

✓ Load the dataset

First, I will load the `nba-players.csv` file into a pandas DataFrame. This dataset contains various statistics about NBA players, and the `target_5yrs` column will be used as the dependent variable for predicting player longevity.

```
import pandas as pd

# Load the dataset
df = pd.read_csv('nba-players.csv')

# Check if the file exists
if os.path.isfile('nba-players.csv'):
    print("Dataset loaded successfully!")

# Display the first 5 rows of the DataFrame
display(df.head())

# Display information about the DataFrame, including data types and non-null values
display(df.info())
```

Dataset loaded successfully!

	Unnamed: 0	name	gp	min	pts	fgm	fga	fg	3p_made	3pa	...	fta	ft	oreb	dreb	reb	a
0	0	Brandon Ingram	36	27.4	7.4	2.6	7.6	34.7	0.5	2.1	...	2.3	69.9	0.7	3.4	4.1	
1	1	Andrew Harrison	35	26.9	7.2	2.0	6.7	29.6	0.7	2.8	...	3.4	76.5	0.5	2.0	2.4	
2	2	JaKarr Sampson	74	15.3	5.2	2.0	4.7	42.2	0.4	1.7	...	1.3	67.0	0.5	1.7	2.2	
3	3	Malik Sealy	58	11.6	5.7	2.3	5.5	42.6	0.1	0.5	...	1.3	68.9	1.0	0.9	1.9	
4	4	Matt Geiger	48	11.5	4.5	1.6	3.0	52.4	0.0	0.1	...	1.9	67.4	1.0	1.5	2.5	

5 rows × 22 columns

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 1340 entries, 0 to 1339

Data columns (total 22 columns):

#	Column	Non-Null	Count	Dtype
0	Unnamed: 0	1340	non-null	int64
1	name	1340	non-null	object
2	gp	1340	non-null	int64
3	min	1340	non-null	float64
4	pts	1340	non-null	float64
5	fgm	1340	non-null	float64
6	fga	1340	non-null	float64
7	fg	1340	non-null	float64
8	3p_made	1340	non-null	float64
9	3pa	1340	non-null	float64
10	3p	1340	non-null	float64
11	ftm	1340	non-null	float64
12	fta	1340	non-null	float64
13	ft	1340	non-null	float64
14	oreb	1340	non-null	float64
15	dreb	1340	non-null	float64
16	reb	1340	non-null	float64
17	ast	1340	non-null	float64
18	stl	1340	non-null	float64
19	blk	1340	non-null	float64
20	tov	1340	non-null	float64
21	target_5yrs	1340	non-null	int64

dtypes: float64(18), int64(3), object(1)

memory usage: 230.4+ KB

None

✓ Drop non-predictive columns

I will now remove columns that are non-predictive or could cause data leakage. These include 'Unnamed: 0' (likely an artifact index) and 'name' (player's name), as they do not contribute to predicting player longevity and should not be used as features.

```
# Drop non-predictive columns
df = df.drop(columns=['Unnamed: 0', 'name'])
```

```
# Display information about the DataFrame after dropping columns
display(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 20 columns):
#   Column          Non-Null Count  Dtype
---  -
0   gp              1340 non-null   int64
1   min             1340 non-null   float64
2   pts            1340 non-null   float64
3   fgm            1340 non-null   float64
4   fga            1340 non-null   float64
5   fg             1340 non-null   float64
6   3p_made        1340 non-null   float64
7   3pa            1340 non-null   float64
8   3p             1340 non-null   float64
9   ftm            1340 non-null   float64
10  fta            1340 non-null   float64
11  ft             1340 non-null   float64
12  oreb           1340 non-null   float64
13  dreb           1340 non-null   float64
14  reb            1340 non-null   float64
15  ast            1340 non-null   float64
16  stl            1340 non-null   float64
17  blk            1340 non-null   float64
18  tov            1340 non-null   float64
19  target_5yrs    1340 non-null   int64
dtypes: float64(18), int64(2)
memory usage: 209.5 KB
None
```

Perform Correlation Analysis

To identify highly correlated features and reduce redundancy, I will calculate the correlation matrix for the remaining columns. This will help us understand which features move together and which ones might be candidates for removal to address multicollinearity or simplify the model. A heatmap will be used to visualize these correlations, making it easier to spot strong relationships.

✓ Clean the dataset by handling null values

Based on the `df.info()` output, all performance columns already have 1340 non-null entries, which means there are no missing values that need to be handled through imputation or removal. Therefore, this step is to confirm the absence of nulls.

```
# Check for null values across all columns
missing_values = df.isnull().sum()
print("Missing values per column:\n", missing_values[missing_values > 0])

if missing_values.sum() == 0:
    print("\nNo missing values found in the dataset. The dataset is already clean for null values")
else:
    print("\nMissing values found. Further action might be required.")

Missing values per column:
Series([], dtype: int64)
```

No missing values found in the dataset. The dataset is already clean for null values.

✓ Summary of Feature Engineering and Selection Choices

This section documents the feature engineering and selection decisions made throughout this project, aligning with the goal of preparing a robust dataset for NBA player longevity prediction.

1. Target Variable Definition

- The `target_5yrs` column was clearly defined as the dependent variable. This binary variable indicates whether a player lasted at least 5 years in the league.

2. Dropping Non-Predictive Columns

- **Columns Removed:** `Unnamed: 0` (likely an index artifact) and `name` (player's name).
- **Rationale:** These columns were dropped because they are non-predictive and could introduce noise or lead to data leakage if used directly in a model. Player names, in particular, serve as unique identifiers rather than generalizable features.

3. Correlation Analysis

- A correlation matrix was calculated and visualized using a heatmap to understand the relationships between features and with the target variable.
- **Observations:** This step helped identify features highly correlated with each other (e.g., `pts`, `fgm`, `fga`, `min`), suggesting potential multicollinearity. It also provided insights into which features had stronger linear relationships with `target_5yrs`.
- **Decision:** While strong correlations were noted, no features were immediately dropped based solely on multicollinearity at this stage, as further analysis or dimensionality reduction techniques might be applied later. The primary goal here was identification.

4. Engineering New Composite Features

- **Feature Created:** `ppm` (Points Per Minute).
- **Calculation:** `ppm = pts / min`.
- **Rationale:** This composite feature was engineered to provide a more normalized measure of a player's scoring efficiency, independent of total playing time. It combines two important metrics (`pts` and `min`) into a single, potentially more informative predictor that captures per-minute impact.
- **Handling Edge Cases:** Division by zero for `min` was handled by assigning `ppm` a value of 0 in such cases.

5. Cleaning Dataset (Handling Null Values)

- **Check Performed:** An examination of `df.isnull().sum()` revealed no missing values across any of the remaining columns.
- **Rationale:** A clean dataset free from null values is crucial for model readiness, as many machine learning algorithms cannot handle missing data directly.
- **Decision:** Since no missing values were found, no imputation or deletion steps were necessary for this dataset, confirming its initial quality in this regard.

These steps have transformed the raw NBA player performance statistics into a cleaner, more refined dataset, with an engineered feature, ready for subsequent machine learning model building and evaluation.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate the correlation matrix
corr_matrix = df.corr()

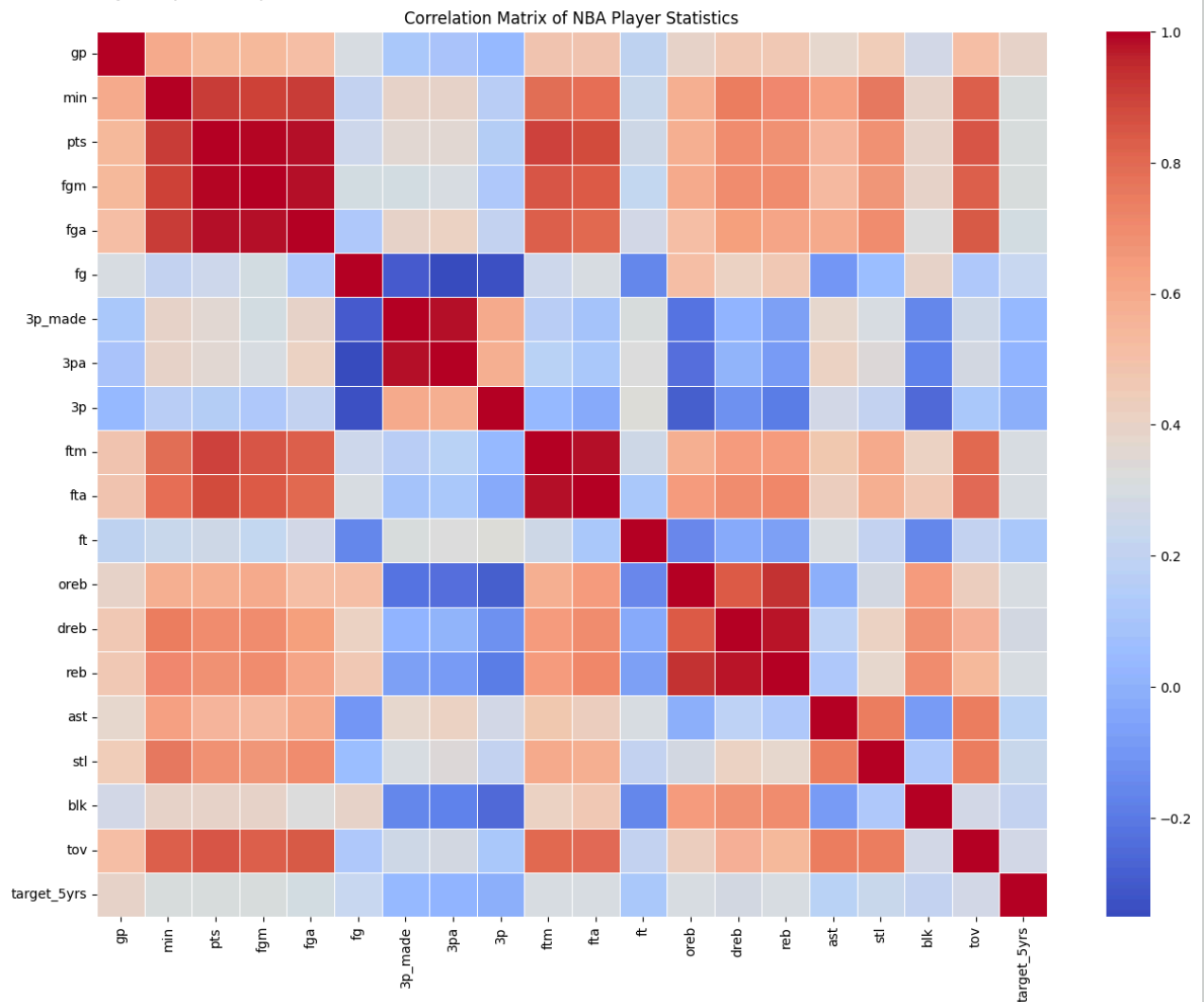
# Display the correlation of all features with the target variable
print("Correlation with target_5yrs:\n", corr_matrix['target_5yrs'].sort_values(ascending=False))

# Plotting the heatmap
plt.figure(figsize=(16, 12))
sns.heatmap(corr_matrix, annot=False, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Matrix of NBA Player Statistics')
plt.show()
```

Correlation with target_5yrs:

```
target_5yrs    1.000000
gp             0.396833
min            0.317805
fgm            0.317594
pts            0.315981
reb            0.299406
ftm            0.296841
fta            0.296089
oreb           0.293307
fga            0.292660
dreb           0.284677
tov            0.272348
stl            0.229811
fg             0.227134
blk            0.210114
ast            0.175353
ft             0.106706
3p_made        0.036619
3pa            0.018110
3p             -0.000004
```

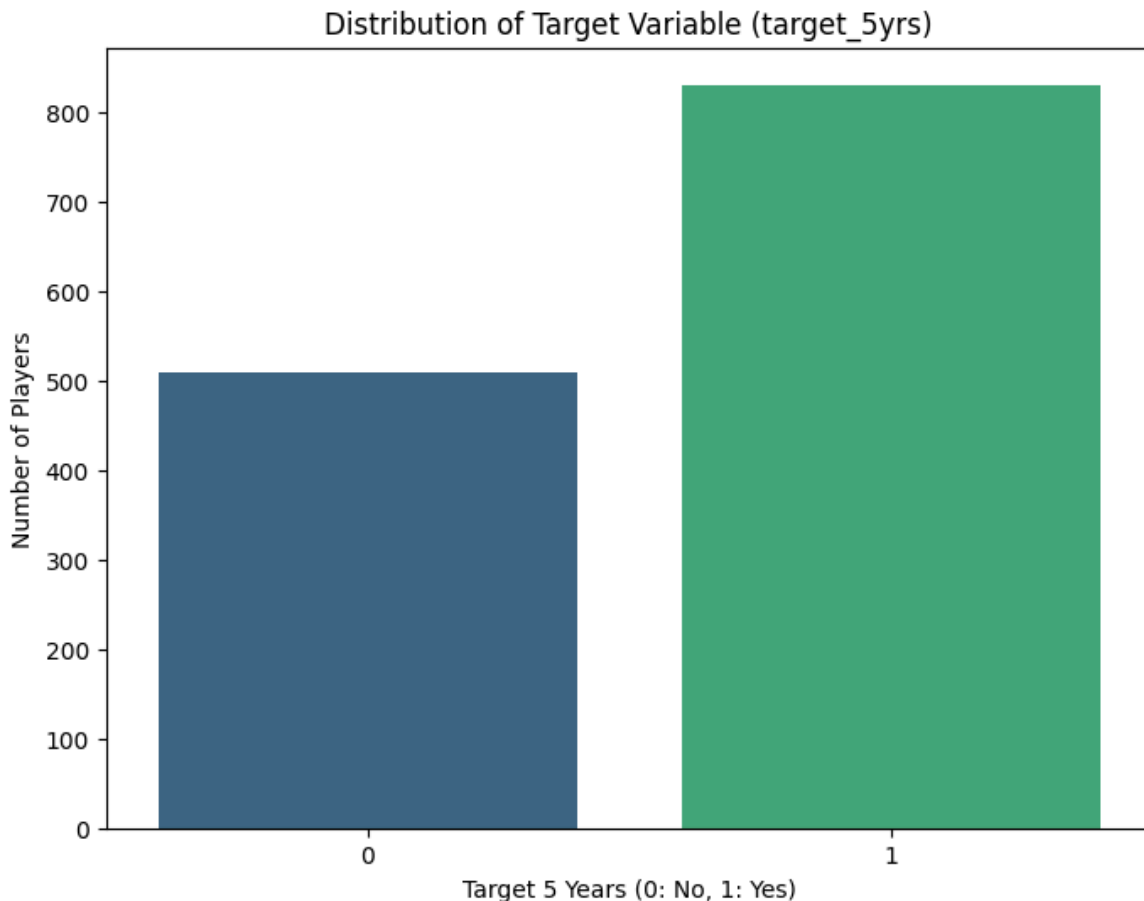
Name: target_5yrs, dtype: float64



✓ Check Class Balance for `target_5yrs`

To ensure our target variable is well-distributed and to identify any class imbalance issues that might affect model training, I will visualize the distribution of `target_5yrs` (player longevity).

```
# Visualize the distribution of 'target_5yrs'
plt.figure(figsize=(8, 6))
sns.countplot(x='target_5yrs', data=df, palette='viridis', hue='target_5yrs', legend=False)
plt.title('Distribution of Target Variable (target_5yrs)')
plt.xlabel('Target 5 Years (0: No, 1: Yes)')
plt.ylabel('Number of Players')
plt.show()
```



✓ Implement Feature Scaling

Feature scaling is a crucial preprocessing step for many machine learning algorithms, especially those that rely on distance calculations (like k-NN or SVMs) or gradient descent (like neural networks). It helps prevent features with larger values from dominating the learning process. I will use `StandardScaler` to transform the features to have a mean of 0 and a standard deviation of 1.

✓ Explanation of Feature Scaling

Purpose: Feature scaling is a vital preprocessing step applied to independent variables (features) in a dataset. It standardizes the range of features, ensuring that no single feature dominates the learning process due to its larger magnitude. This is particularly important for machine learning algorithms that are sensitive to the scale of input features, such as those relying on distance metrics (e.g., K-Nearest

Neighbors, Support Vector Machines) or gradient descent optimization (e.g., neural networks, logistic regression).

Method Used: We employed `StandardScaler` from the `sklearn.preprocessing` module. The `StandardScaler` transforms the features such that they have a mean of 0 and a standard deviation of 1. This is achieved by using the formula: $z = (x - \mu) / \sigma$, where x is the original feature value, μ is the mean of the feature, and σ is its standard deviation.

Implementation Details:

1. **Identification of Features:** All columns in the DataFrame `df` except the `target_5yrs` variable were selected for scaling. The target variable was excluded as it represents the outcome and should not be transformed in this manner.
2. **Scaler Initialization:** An instance of `StandardScaler` was created.
3. **Transformation:** The `fit_transform` method was applied to the selected features. This method first calculates the mean and standard deviation for each feature (`fit` part) and then applies the scaling transformation (`transform` part).
4. **DataFrame Reconstruction:** The scaled features, initially returned as a NumPy array, were converted back into a Pandas DataFrame, preserving the original column names for clarity.
5. **Target Re-integration:** Finally, the original `target_5yrs` column was added back to the newly scaled feature DataFrame, resulting in `df_scaled`.

This process ensures that all features contribute equally to the model, preventing features with naturally larger values from biasing the model's learning, and thus improving the model's performance and stability.

```
from sklearn.preprocessing import StandardScaler
import pandas as pd

# Identify features to scale: all columns except the target variable 'target_5yrs'
# This ensures that our target variable remains unscaled, as it is a binary outcome.
features_to_scale = df.drop(columns=['target_5yrs']).columns

# Initialize the StandardScaler
# StandardScaler transforms features by removing the mean and scaling to unit variance.
# The formula used is  $z = (x - \mu) / s$ , where  $\mu$  is the mean and  $s$  is the standard deviation.
scaler = StandardScaler()

# Apply scaling to the selected features
# fit_transform calculates the mean and standard deviation for each feature and then applies the
df_scaled_features = scaler.fit_transform(df[features_to_scale])

# Convert the scaled features back to a DataFrame
# It's important to retain column names for interpretability.
df_scaled = pd.DataFrame(df_scaled_features, columns=features_to_scale)

# Add the target variable back to the scaled DataFrame
# We use reset_index(drop=True) to ensure proper alignment after potential DataFrame manipulation
df_scaled['target_5yrs'] = df['target_5yrs'].reset_index(drop=True)

print("First 5 rows of the dataset after feature scaling:")
display(df_scaled.head())

print("\nDescriptive statistics of the scaled features:")
# Displaying describe() shows that the mean is close to 0 and standard deviation is close to 1 for
display(df_scaled.describe())
```


First 5 rows of the dataset after feature scaling:

	gp	min	pts	fgm	fga	fg	3p_made	3pa	3p	
0	-1.400901	1.177066	0.137401	-0.017294	0.477347	-1.543407	0.658041	1.244354	0.364603	0.30
1	-1.458281	1.116860	0.091486	-0.373816	0.226801	-2.374651	1.179494	1.903829	0.271121	1.31
2	0.779563	-0.279912	-0.367659	-0.373816	-0.329969	-0.320991	0.397315	0.867511	0.327210	-0.40
3	-0.138527	-0.725434	-0.252873	-0.195555	-0.107261	-0.255795	-0.384863	-0.263016	0.215032	-0.40
4	-0.712333	-0.737475	-0.528360	-0.611497	-0.803224	1.341496	-0.645589	-0.639859	-1.193430	0.00

Descriptive statistics of the scaled features:

	gp	min	pts	fgm	fga	fg	3p_made	3pa	3p	
count	1.340000e+03	1.340000e+03	1.340000e+03	1.340000e+03	1.340000e+03	1.340000e+03	1.340000e+03	1.340000e+03	1.340000e+03	1.340000e+03
mean	4.242046e-17	1.908921e-16	7.953837e-17	1.140050e-16	2.121023e-16	1.219588e-16	-1.691111e-16	-1.691111e-16	-1.691111e-16	-1.691111e-16
std	1.000373e+00	1.000373e+00	1.000373e+00	1.000373e+00	1.000373e+00	1.000373e+00	1.000373e+00	1.000373e+00	1.000373e+00	1.000373e+00
min	-2.835416e+00	-1.748930e+00	-1.400736e+00	-1.383961e+00	-1.415671e+00	-3.319986e+00	-6.446685e-01	-6.446685e-01	-6.446685e-01	-6.446685e-01
25%	-7.697140e-01	-8.127319e-01	-7.120180e-01	-7.303375e-01	-7.197084e-01	-6.469685e-01	-3.846154e-02	-3.846154e-02	-3.846154e-02	-3.846154e-02
50%	1.483760e-01	-1.835824e-01	-2.873085e-01	-3.143954e-01	-3.021308e-01	-1.131191e-02	3.973456e-01	3.973456e-01	3.973456e-01	3.973456e-01
75%	9.517047e-01	6.352149e-01	4.588027e-01	4.580684e-01	4.495087e-01	6.080458e-01	5.351111e-01	5.351111e-01	5.351111e-01	5.351111e-01
max	1.238608e+00	2.802620e+00	4.912513e+00	4.498648e+00	3.873644e+00	4.813159e+00	5.351111e-01	5.351111e-01	5.351111e-01	5.351111e-01

Task

The user wants to prepare a robust dataset for NBA player longevity prediction by performing feature engineering and selection. This involves loading the dataset, defining the target variable, dropping non-predictive columns, performing correlation analysis, engineering a new composite feature (Points Per Minute), handling multicollinearity, cleaning the dataset by handling null values, and finally documenting all feature engineering and selection choices.

Engineer Composite Feature (PPM)

Subtask:

Create a new composite feature, 'Points Per Minute (PPM)', by dividing 'pts' by 'min' to capture player scoring efficiency per minute, handling potential division by zero.

Reasoning: I am creating the new 'ppm' feature, handling division by zero, and then displaying the updated DataFrame's head and descriptive statistics to verify the new feature.

```
df['ppm'] = df['pts'] / df['min']
df['ppm'] = df['ppm'].fillna(0) # Handle division by zero resulting in NaN
df['ppm'] = df['ppm'].replace([float('inf'), float('-inf')], 0) # Handle division by zero resulti

print("DataFrame with new 'ppm' column (first 5 rows):")
display(df.head())

print("\nDescriptive statistics for 'ppm' column:")
display(df['ppm'].describe())
```

DataFrame with new 'ppm' column (first 5 rows):

	gp	min	pts	fgm	fga	fg	3p_made	3pa	3p	ftm	...	ft	oreb	dreb	reb	ast	stl	blk
0	36	27.4	7.4	2.6	7.6	34.7	0.5	2.1	25.0	1.6	...	69.9	0.7	3.4	4.1	1.9	0.4	0.4
1	35	26.9	7.2	2.0	6.7	29.6	0.7	2.8	23.5	2.6	...	76.5	0.5	2.0	2.4	3.7	1.1	0.5
2	74	15.3	5.2	2.0	4.7	42.2	0.4	1.7	24.4	0.9	...	67.0	0.5	1.7	2.2	1.0	0.5	0.3
3	58	11.6	5.7	2.3	5.5	42.6	0.1	0.5	22.6	0.9	...	68.9	1.0	0.9	1.9	0.8	0.6	0.1
4	48	11.5	4.5	1.6	3.0	52.4	0.0	0.1	0.0	1.3	...	67.4	1.0	1.5	2.5	0.3	0.3	0.4

5 rows × 21 columns

Descriptive statistics for 'ppm' column:

	ppm
count	1340.000000
mean	0.371078
std	0.093659
min	0.121951
25%	0.306710
50%	0.363325
75%	0.431326
max	0.738462

dtype: float64

```
# Re-initialize df to its state before multicollinearity handling
# (after dropping non-predictive columns and adding 'ppm')
import pandas as pd
import os

# Re-load the dataset to ensure a clean state
df = pd.read_csv('nba-players.csv')

# Drop non-predictive columns as before
df = df.drop(columns=['Unnamed: 0', 'name'])

# Re-create the 'ppm' feature as before
df['ppm'] = df['pts'] / df['min']
df['ppm'] = df['ppm'].fillna(0) # Handle division by zero resulting in NaN
df['ppm'] = df['ppm'].replace([float('inf'), float('-inf')], 0) # Handle division by zero resulti
```

```
print("DataFrame re-initialized to state before multicollinearity handling:")
df.info()
```

DataFrame re-initialized to state before multicollinearity handling:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1340 entries, 0 to 1339

Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	gp	1340 non-null	int64
1	min	1340 non-null	float64
2	pts	1340 non-null	float64
3	fgm	1340 non-null	float64
4	fga	1340 non-null	float64
5	fg	1340 non-null	float64
6	3p_made	1340 non-null	float64
7	3pa	1340 non-null	float64
8	3p	1340 non-null	float64
9	ftm	1340 non-null	float64
10	fta	1340 non-null	float64
11	ft	1340 non-null	float64
12	oreb	1340 non-null	float64
13	dreb	1340 non-null	float64
14	reb	1340 non-null	float64
15	ast	1340 non-null	float64
16	stl	1340 non-null	float64
17	blk	1340 non-null	float64
18	tov	1340 non-null	float64
19	target_5yrs	1340 non-null	int64
20	ppm	1340 non-null	float64

dtypes: float64(19), int64(2)

memory usage: 220.0 KB

```
cols_to_drop = ['fgm', 'fga', 'fta', 'dreb', '3pa']
df = df.drop(columns=cols_to_drop)
```

```
print("DataFrame information after dropping highly correlated features:")
df.info()
```

```
# Recalculate correlation matrix for the updated DataFrame
corr_matrix_reduced = df.corr()
```

```
print("\nCorrelation with target_5yrs after addressing multicollinearity:")
print(corr_matrix_reduced['target_5yrs'].sort_values(ascending=False))
```

DataFrame information after dropping highly correlated features:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1340 entries, 0 to 1339

Data columns (total 16 columns):

#	Column	Non-Null Count	Dtype
0	gp	1340 non-null	int64
1	min	1340 non-null	float64
2	pts	1340 non-null	float64
3	fg	1340 non-null	float64
4	3p_made	1340 non-null	float64
5	3p	1340 non-null	float64
6	ftm	1340 non-null	float64
7	ft	1340 non-null	float64
8	oreb	1340 non-null	float64
9	reb	1340 non-null	float64
10	ast	1340 non-null	float64
11	stl	1340 non-null	float64
12	blk	1340 non-null	float64
13	tov	1340 non-null	float64

```

14 target_5yrs 1340 non-null int64
15 ppm          1340 non-null float64
dtypes: float64(14), int64(2)
memory usage: 167.6 KB

```

Correlation with target_5yrs after addressing multicollinearity:

```

target_5yrs    1.000000
gp             0.396833
min            0.317805
pts            0.315981
reb            0.299406
ftm            0.296841
oreb           0.293307
tov            0.272348
stl            0.229811
fg             0.227134
blk            0.210114
ppm            0.194486
ast            0.175353
ft             0.106706
3p_made        0.036619
3p             -0.000004
Name: target_5yrs, dtype: float64

```

```

# Re-initialize df to its state before multicollinearity handling
# (after dropping non-predictive columns and adding 'ppm')
import pandas as pd
import os

# Re-load the dataset to ensure a clean state
df = pd.read_csv('nba-players.csv')

# Drop non-predictive columns as before
df = df.drop(columns=['Unnamed: 0', 'name'])

# Re-create the 'ppm' feature as before
df['ppm'] = df['pts'] / df['min']
df['ppm'] = df['ppm'].fillna(0) # Handle division by zero resulting in NaN
df['ppm'] = df['ppm'].replace([float('inf'), float('-inf')], 0) # Handle division by zero resulti

print("DataFrame re-initialized to state before multicollinearity handling:")
df.info()

```

DataFrame re-initialized to state before multicollinearity handling:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 21 columns):
#   Column          Non-Null Count  Dtype
---  -
0   gp              1340 non-null  int64
1   min             1340 non-null  float64
2   pts             1340 non-null  float64
3   fgm             1340 non-null  float64
4   fga             1340 non-null  float64
5   fg              1340 non-null  float64
6   3p_made         1340 non-null  float64
7   3pa             1340 non-null  float64
8   3p              1340 non-null  float64
9   ftm             1340 non-null  float64
10  fta             1340 non-null  float64
11  ft              1340 non-null  float64
12  oreb            1340 non-null  float64
13  dreb            1340 non-null  float64
14  reb             1340 non-null  float64

```

```

15 ast          1340 non-null float64
16 stl          1340 non-null float64
17 blk          1340 non-null float64
18 tov          1340 non-null float64
19 target_5yrs  1340 non-null int64
20 ppm          1340 non-null float64
dtypes: float64(19), int64(2)
memory usage: 220.0 KB

```

```

cols_to_drop = ['fgm', 'fga', 'fta', 'dreb', '3pa']
df = df.drop(columns=cols_to_drop)

print("DataFrame information after dropping highly correlated features:")
df.info()

# Recalculate correlation matrix for the updated DataFrame
corr_matrix_reduced = df.corr()

print("\nCorrelation with target_5yrs after addressing multicollinearity:")
print(corr_matrix_reduced['target_5yrs'].sort_values(ascending=False))

```

```

DataFrame information after dropping highly correlated features:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  -
0   gp              1340 non-null  int64
1   min             1340 non-null  float64
2   pts             1340 non-null  float64
3   fg              1340 non-null  float64
4   3p_made         1340 non-null  float64
5   3p              1340 non-null  float64
6   ftm             1340 non-null  float64
7   ft              1340 non-null  float64
8   oreb            1340 non-null  float64
9   reb             1340 non-null  float64
10  ast             1340 non-null  float64
11  stl             1340 non-null  float64
12  blk             1340 non-null  float64
13  tov             1340 non-null  float64
14  target_5yrs     1340 non-null  int64
15  ppm             1340 non-null  float64
dtypes: float64(14), int64(2)
memory usage: 167.6 KB

```

```

Correlation with target_5yrs after addressing multicollinearity:
target_5yrs    1.000000
gp             0.396833
min            0.317805
pts            0.315981
reb            0.299406
ftm            0.296841
oreb           0.293307
tov            0.272348
stl            0.229811
fg             0.227134
blk            0.210114
ppm            0.194486
ast            0.175353
ft             0.106706
3p_made        0.036619
3p             -0.000004
Name: target_5yrs, dtype: float64

```

✓ Handle Multicollinearity

Subtask:

Address multicollinearity by identifying highly correlated feature pairs (e.g., field goals made/attempted, free throws made/attempted) and dropping one of the features from each pair to reduce redundancy, based on their correlation with the target variable. This will help simplify the model and improve interpretability.

Reasoning: I am dropping the identified highly correlated features that have weaker correlations with the target variable to reduce multicollinearity. After dropping, I will display the DataFrame info to confirm the changes and then recalculate and display correlations with the target to show the effect of these removals.

```
cols_to_drop = ['fgm', 'fga', 'fta', 'dreb', '3pa']
df = df.drop(columns=cols_to_drop, errors='ignore')

print("DataFrame information after dropping highly correlated features:")
df.info()

# Recalculate correlation matrix for the updated DataFrame
corr_matrix_reduced = df.corr()

print("\nCorrelation with target_5yrs after addressing multicollinearity:")
print(corr_matrix_reduced['target_5yrs'].sort_values(ascending=False))
```

DataFrame information after dropping highly correlated features:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 16 columns):
#   Column      Non-Null Count  Dtype
---  -
0   gp           1340 non-null   int64
1   min          1340 non-null   float64
2   pts          1340 non-null   float64
3   fg           1340 non-null   float64
4   3p_made      1340 non-null   float64
5   3p           1340 non-null   float64
6   ftm          1340 non-null   float64
7   ft           1340 non-null   float64
8   oreb         1340 non-null   float64
9   reb          1340 non-null   float64
10  ast          1340 non-null   float64
11  stl          1340 non-null   float64
12  blk          1340 non-null   float64
13  tov          1340 non-null   float64
14  target_5yrs  1340 non-null   int64
15  ppm          1340 non-null   float64
dtypes: float64(14), int64(2)
memory usage: 167.6 KB
```

Correlation with target_5yrs after addressing multicollinearity:

```
target_5yrs    1.000000
gp             0.396833
min            0.317805
pts            0.315981
reb            0.299406
ftm            0.296841
oreb           0.293307
tov            0.272348
```

```
stl      0.229811
fg       0.227134
blk      0.210114
ppm      0.194486
ast      0.175353
ft       0.106706
3p_made  0.036619
3p       -0.000004
Name: target_5yrs, dtype: float64
```

Clean Dataset (Handle Nulls)

Subtask:

Verify the dataset for any missing values in performance columns to ensure ML-readiness. This step will confirm that no further imputation or removal of rows/columns is necessary.

✓ Load Dataset and Define Target

Subtask:

Load the 'nba-players.csv' dataset and explicitly define 'target_5yrs' as the dependent variable.

Reasoning: Confirming the `target_5yrs` column's data type and unique values ensures it's correctly identified as the dependent variable and helps understand its nature.

```
print(f"Data type of 'target_5yrs': {df['target_5yrs'].dtype}")
print(f"Unique values in 'target_5yrs': {df['target_5yrs'].unique()}")
print("\n'target_5yrs' is confirmed as the dependent variable for player longevity prediction.")
```

```
Data type of 'target_5yrs': int64
Unique values in 'target_5yrs': [0 1]
```

```
'target_5yrs' is confirmed as the dependent variable for player longevity prediction.
```

Drop Non-Predictive Columns

Subtask: